

MISSION RISER

Git & GitHub — Practical Notes

A practical study guide covering the Git concepts and commands learned for the college website internship preparation.

1. What is Git?

Git is a version control system (VCS). It tracks changes made to files over time and lets developers maintain versions, collaborate, and recover from mistakes.

```
Version 1 → Initial website
Version 2 → Added navigation
Version 3 → Added Year 4
Version 4 → Updated syllabus
```

Verbal answer: Git is a distributed version control system used to track changes, maintain different versions of a project, and support collaboration.

2. Git vs GitHub

Git is the version-control tool running on your computer. **GitHub** is an online platform for hosting Git repositories and collaboration.

```
Your computer → Git → GitHub
```

Remember: Git does not automatically publish a website. Deployment is a separate process.

3. Repository

A **repository (repo)** is a project managed by Git.

```
PortfolioWebsite/
■■■ index.html
■■■ style.css
■■■ .git/
```

The hidden **.git** directory contains Git's internal data and history. Normally, you should not manually modify it.

4. git init

```
git init
```

Initializes Git in the current directory and creates the **.git** directory. Run it inside the project's root folder.

5. Git identity

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

Git records the author of each commit. The global option applies the identity to repositories on that computer.

6. git status

```
git status
```

Shows the current state of the repository: modified files, untracked files, staged changes, and whether the working tree is clean.

```
nothing to commit, working tree clean
```

This means there are no uncommitted changes.

7. git add

```
git add .  
git add index.html
```

git add . stages all changes. **git add index.html** stages only that file. Staging prepares changes for the next commit.

8. Staging area

```
Working directory  
  ↓ git add  
Staging area  
  ↓ git commit  
Repository history
```

The staging area lets you choose exactly which changes should go into the next commit.

9. git commit

```
git commit -m "Added Year 4 page"
```

A commit creates a saved snapshot of the staged changes. The message describes what was changed and is mainly for humans.

10. Understanding a commit

```
[master b1aca58] changed the head text  
1 file changed, 1 insertion(+), 1 deletion(-)
```

master = branch name. **b1aca58** = shortened commit ID/hash. The message describes the commit. The insertion/deletion counts summarize line changes.

11. git log

```
git log  
git log --oneline
```

Shows commit history. The compact form is useful for quickly seeing commit IDs and messages.

12. HEAD

```
b1aca58 (HEAD -> master) changed the head text
```

HEAD represents your current position in Git history. In this example, HEAD is pointing to the latest commit on master.

13. git diff

```
git diff
```

Shows the exact unstaged changes in your working files.

```
-<h1>Portfolio of Nived</h1>
+<h1>Portfolio of Nived Website</h1>
```

A plus sign means a line was added; a minus sign means a line was removed. Good habit: inspect with git diff before committing important changes.

14. git restore

```
git restore index.html
```

Restores the specified file to its last committed version, discarding its current uncommitted changes.

Warning: This can permanently discard uncommitted work. Use it carefully.

15. Branches

A **branch** is a separate line of development. It lets you work on a feature or update without directly changing the main version.

```
master
■
■■■ add-year4
```

A commit made while on add-year4 belongs to that branch and does not automatically appear on master.

16. git branch

```
git branch
```

Lists branches. The * marks the branch you are currently on.

```
* add-year4
master
```

17. Creating a branch

```
git switch -c add-year4
```

Creates a new branch and immediately switches to it. You can also create without switching using git branch add-year4.

18. Switching branches

```
git switch master
git switch add-year4
```

Switches your working directory to an existing branch. The files you see can change according to the selected branch.

19. Merge

```
git switch master
git merge add-year4
```

This brings the changes from add-year4 into master.

Key rule: The branch you are currently ON is the branch that receives the merge.

```
On master + merge add-year4 → add-year4 into master
On add-year4 + merge master → master into add-year4
```

20. Merging multiple branches

```
git switch master
git merge add-year4
git merge add-pdf
git merge update-contact
```

Branches can be merged one at a time. The finished branches can remain or be deleted later if the team's workflow allows it.

21. Merge conflicts

A merge conflict occurs when Git cannot automatically decide how to combine changes, often because two branches modified the same part of a file.

```
<<<<<< HEAD
Your version
=====
Other version
>>>>>> branch
```

Resolve the file manually, remove the conflict markers, then stage and commit the resolved file.

22. git revert

```
git revert <commit-id>
```

Reverts an already committed change by creating a new commit that reverses the earlier commit. The original commit remains in history.

```
A → B → C (bad) → D (revert C)
```

For shared projects, revert is generally safer than rewriting shared history.

23. restore vs revert

```
Uncommitted mistake:
Edit → git restore → discard local change
```

```
Committed mistake:
Commit → git revert → new commit reverses it
```

Restore = discard uncommitted file changes. Revert = reverse an existing commit with a new commit.

24. Remote repository

A remote is another copy of the repository, commonly hosted online. A typical remote name is **origin**.

```
Your computer → Git → GitHub repository
```

25. GitHub repository

```
git remote add origin <repository-url>
git remote -v
```

The first command connects your local repository to the remote. The second checks the configured remote URL.

26. git push

```
git push
```

Uploads your local commits to the remote repository.

```
Edit → git add . → git commit -m "... " → git push → GitHub
```

Important: pushing to GitHub does not automatically mean the live college website is updated. That depends on the deployment process.

27. git pull

```
git pull
```

Gets the latest changes from the remote repository and integrates them into your current local branch.

```
GitHub → git pull → Your computer
```

28. master vs main

master and **main** are simply branch names. Many newer repositories use main, while older ones may use master.

```
git branch -M main
```

29. Practical website workflow

```
git pull
```

```
git switch -c update-academics
```

```
# Edit and test website
```

```
git diff
git add .
git commit -m "Updated academic year"
git push
```

```
# Review / approval / merge / deployment
```

The exact deployment workflow depends on the college's infrastructure. Do not assume that git push directly changes the public website.

30. Mission RISER command sheet

```
git init
git status
git add .
git add filename
git commit -m "message"
git diff
git log --oneline
git restore filename
git branch
git switch -c branch-name
git switch branch-name
git merge branch-name
git revert <commit-id>
git remote add origin <repository-url>
git remote -v
git push
git pull
```

Priority: Know what each command does and when you would use it. You do not need to memorize every advanced Git command for an internship-level verbal test.

31. Verbal test quick answers

What is Git? A version control system used to track changes and maintain project history.

What is a commit? A saved snapshot of staged changes with a descriptive message.

What is a branch? A separate line of development used to work on changes without directly modifying the main version.

What is merge? Combining changes from one branch into another; the current branch receives the merge.

What is revert? Creating a new commit that reverses an earlier commit.

Git vs GitHub? Git tracks versions locally; GitHub hosts Git repositories online and supports collaboration.

What if you make a mistake? Check the diff/history, use restore for uncommitted mistakes or revert for an already committed shared change, and follow the team's deployment/rollback procedure.